Ca' Foscari University of Venice

Development Methodology

Matteo Zanioli

matteo.zanioli@unive.it

Ca' Foscari University of Venice

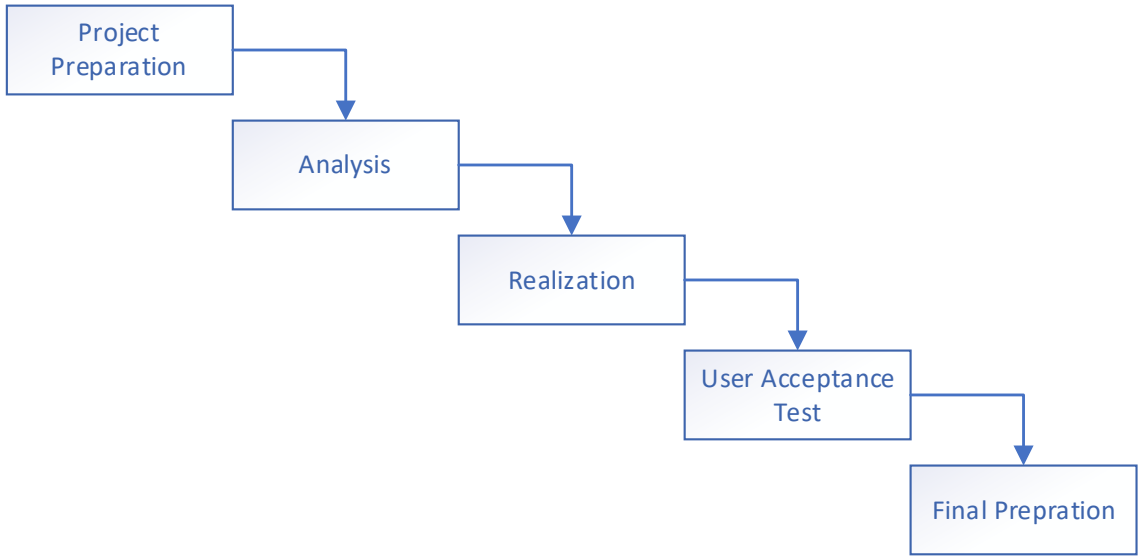
A little bit of theory

- One upon a time there was a methodology called waterfall... and it is still alive!

- Introduced in '70, it is based on a sequences of phases: before to move to the next phase, you need to complete the previous one; moreover, you cannot go back

Ca' Foscari University of Venice

Waterfall methodology



Royce, W. W. (1970). *Managing the Development of Large Software Systems*. Proceedings of IEEE WESCON, 1-9.

Implementation steps

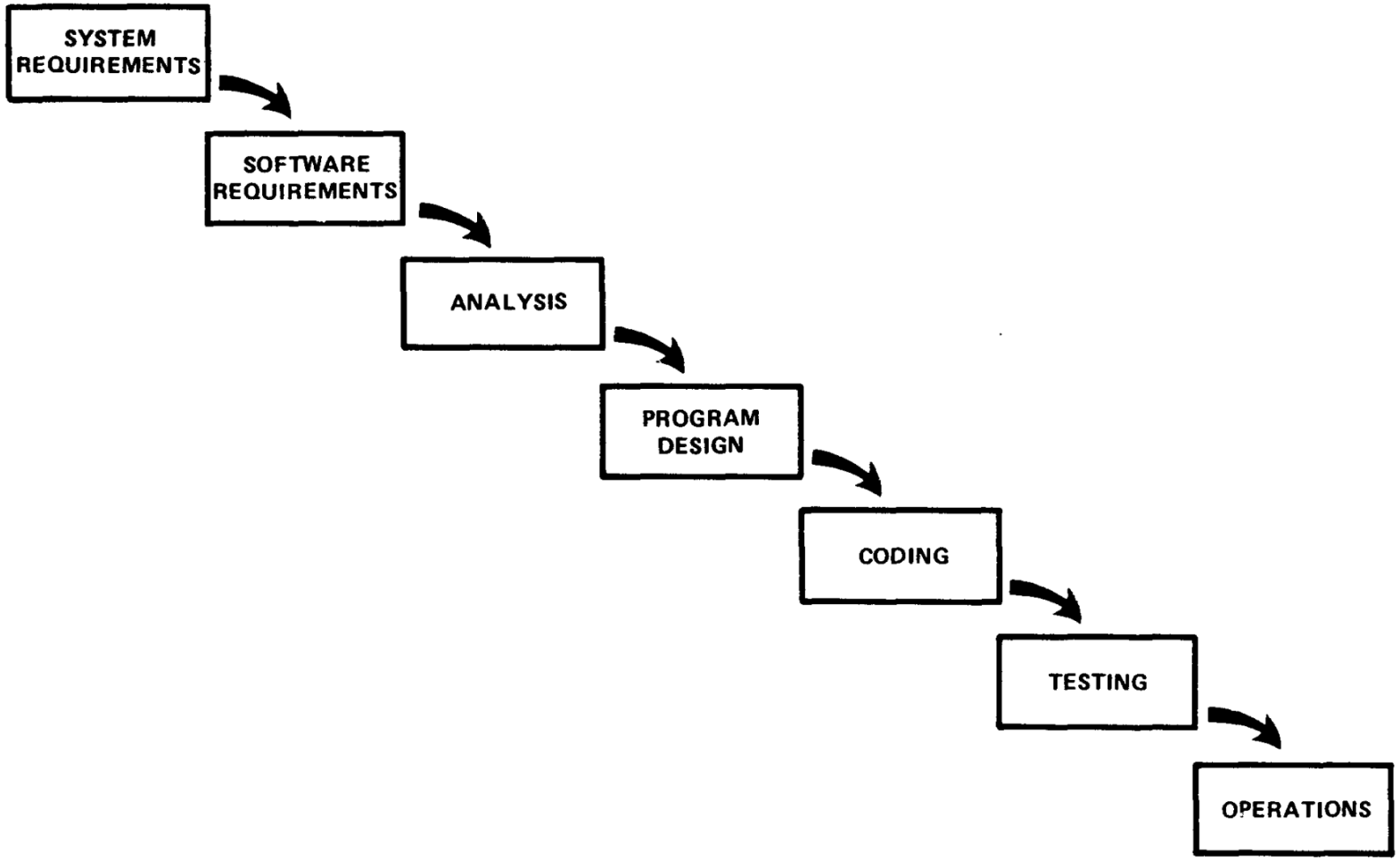


Figure 2. Implementation steps to develop a large computer program for delivery to a customer.

Documentation for each step

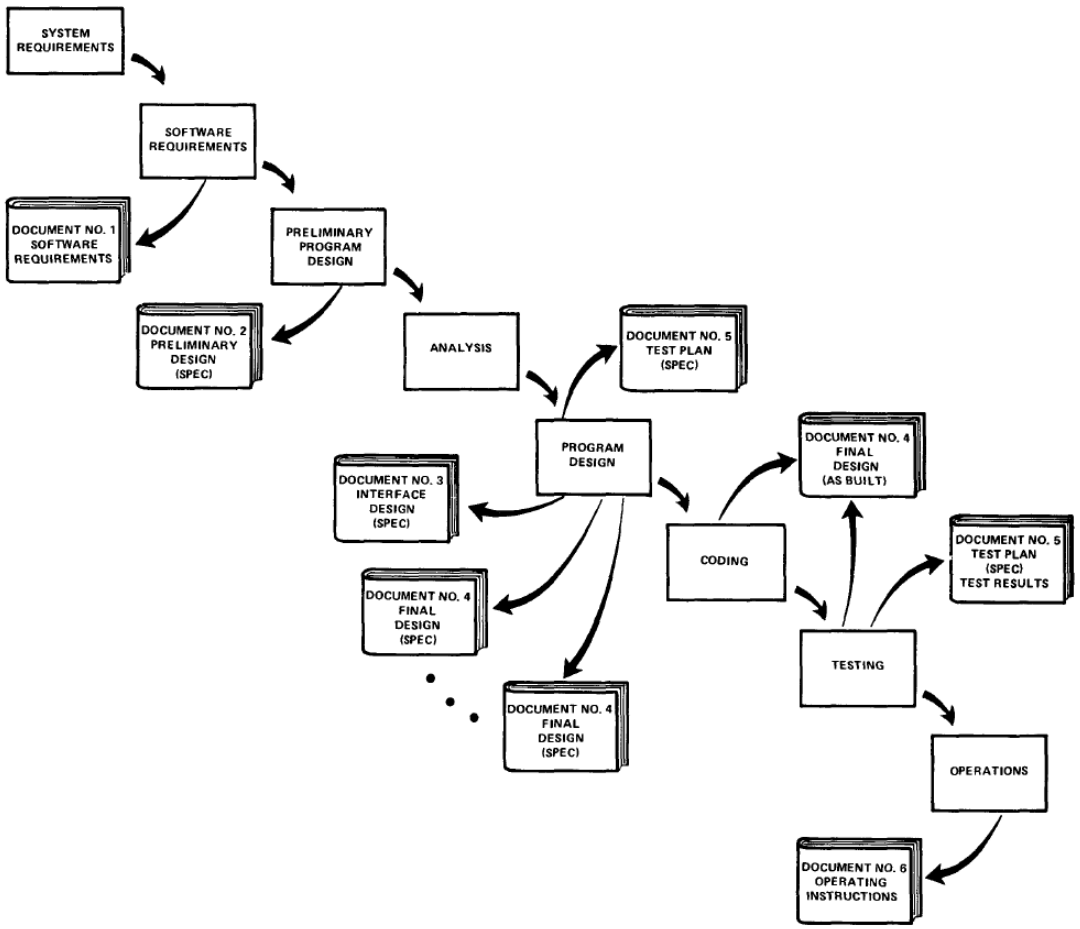


Figure 6. Step 2: Insure that documentation is current and complete – at least six uniquely different documents are required.

Customer involvement

- For some reason what a software design is going to do is subject to wide interpretation even after previous agreement.
- It is important to involve the customer in a formal way so that he has committed himself at earlier points before final delivery.
- To give the contractor free rein between requirement definition and operation is inviting trouble.

Ca' Foscari University of Venice

Waterfall methodology – pros and cons

1. Go to wooclap.com
2. Enter the event code in the top banner

Event code
KMPMIL

Waterfall methodology – pros and cons

Pros

- **Clear Structure**: Waterfall follows a linear and sequential approach, which provides a clear, easy-to-understand structure with distinct phases like requirements, design, implementation, testing, and maintenance
- **Well-documented Process**: Each phase is completed before moving on to the next, leading to comprehensive documentation at every step. This is particularly useful in regulated industries or when knowledge transfer is important
- **Easy to Manage**: Since Waterfall has a defined scope, timeline, and deliverables from the start, it's relatively easier to manage, especially in projects with fixed requirements.
- **Good for Simple or Short-term Projects**: Waterfall can work well for projects with well-defined requirements that are unlikely to change, or for small, straightforward projects with clear goals.

Cons

- **Lack of Flexibility**: Waterfall doesn't easily accommodate changes once a phase has been completed, making it difficult to adapt to new requirements or feedback after the initial planning stage
- **Late Testing**: Testing occurs only after the development phase is complete, which means bugs or issues are often discovered late in the process, making them more costly and time-consuming to fix
- **Risk of Misalignment**: Since the customer doesn't see the product until late in the process, there's a risk of misalignment between the customer's expectations and the final product, especially if requirements were misunderstood at the start
- **Longer Delivery Times**: The linear nature of Waterfall means that you need to wait until the entire project is finished before delivering any functional software, which can delay feedback and value delivery.

Agile, a response for the limitation of Waterfall

- 1950 – Lean Manufactoring implemented in Toyota - https://amzn.eu/d/4wFnJEF
- 1990 – Devs an PMs began experimenting more flexible and iterative methods to tackle software development challenges
- 2001 – 17 devs and industry leaders define the core principles of a new approach and create the Agile Manifesto - https://agilemanifesto.org/
- Agile has lead to development of several frameworks, including Scrum, Kanban, Extreme Programming and others

Everything started in the automotive industry

- Between 1948 and 1975, Taiichi Ohno and Eiji Toyoda introduce – in Toyota – the Toyota Production System (TPS).
- TPS introduce many main concepts:
  - "Muda": waste reduction (identify and eliminating any activity or resource that does not add value to the final product)
  - "Kaizen": Continuous Improvement, small incremental improvements to processes
  - Just-in-time: pull production system, where production is driven by actual customer demand rather than forecasted demand, reducing inventory costs and improving responsiveness
  - "Jidoka": automation with a human "touch" - workers are involved in improving production processes and rely on standardized work to ensure consistency and quality
- These concepts was revolutionary at that time; thanks to TPS, Toyota became one of the global leader for automotive industry and the model for "lean manufacturing"

Ca' Foscari University of Venice

Everything started in the automotive industry

- The term "Lean Manufactoring" rise in 1990, when the TPS was move outside the automotive industry
- The term "lean" is used to describe the focus on eliminating waste, improving efficiency and archieving more with fewer resources
- Taiichi Ohno implemented a visual tool to manage and control work-in-progress (WIP): Kanban
- Kanban: a board with different columns, in general "To Do", "In Progress" and "Done"
- Kanban works with the pull system: tasks are pulled into the workflow only when capacity becomes available, avoiding overloading the team

Ca' Foscari University of Venice

Example of lean thinking

Ca' Foscari University of Venice

Examples of lean thiking

Virginia Mason Franciscan Health

amazon

ZARA

BOEING

Lean Thinking applied to software development

- If you are interested in the topic: "Lean Software Development" (Mary and Tom Poppendieck) - https://amzn.to/3XCYdE5
- Core topics:
  - Lean Thinking in software development
  - Work's flow optimization
  - Continuous improvement for increase the value

Agile Manifesto

Values

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

Principles

- 12 principles (check the website for the complete lists)
- Some principles:
  - Satisfying the customer through early and continuous delivery of valuable software
  - Welcoming changing requirements, even late in development
  - Continuous collaboration with the customer
  - Working software as the primary measure of progress
  - Self-organizing teams to create the best architecture and design

Agile Manifesto

- The core idea of Agile is to break the work into **iterations** or **sprints**, which are short, repetitive cycles that allow the release of functional software increments. This approach enables teams to quickly adapt to changes and continuously receive feedback from the client, improving the quality and relevance of the final product.

Lean vs Agile

Lean	Agile
Muda – waste reduction	Optimize the time in order to work only activities that have a high value
Kaizen – continuous improvement	Continuous improvements in order to improve process and the product/software + continuous deployment
Just-in-time	Work only the "stories" that the customer asked for
Jidoka - workers are involved in improving production	Automate process and self organization

Is the agile the final answer?

I could say yes, but...

Ca' Foscari University of Venice

Is the agile the final answer?

...it will not be the truth

Ca' Foscari University of Venice

Is the agile the final answer?

Is the agile the final answer?

As usual, the answer is:

depends

Real case

- pure Agile methodology works quite well if your team works as software house to your own software
- if you extend someone else software could be not the best solution
- if you have a customer who does not have a strict deadline the risk is that you never bring your software live (this is a real case – more common than you might think)

Agile

- come back to some slides before: "Agile has lead to development of several frameworks, including Scrum, Kanban, Extreme Programming and others"

- Now, let's see some of this frameworks.

Ca' Foscari University of Venice

Scrum methodology

- Scrum is based on iterative and incremental cycles called **sprints**, with the goal of delivering value quickly and frequently.
- It is designed to adapt to constant changes and encourages collaboration and continuous improvement

Scrum methodology

- Roles
  - **Product Owner**: Responsible for managing the Product Backlog (the list of features or goals to be achieved) and setting priorities. They represent the voice of the customer and ensure the team is working on the right things.
  - **Scrum Master**: Acts as a facilitator of the process, ensuring Scrum is implemented correctly, removing obstacles that block the team, and guiding continuous improvement. The Scrum Master is not a manager but a "servant leader" for the team.
  - **Development Team**: A group of professionals who work together to achieve the goals of a sprint. The team is cross-functional, meaning it includes all the skills necessary to complete the work. They are self-organizing and autonomous.
- Artifacts
  - **Product Backlog**: A prioritized list of all requirements and features that need to be developed. Managed by the Product Owner, it is continuously updated and refined.
  - **Sprint Backlog**: A subset of the Product Backlog that the team commits to completing during a sprint. It contains specific tasks and goals for the current cycle.
  - **Product Increment**: At the end of each sprint, the team delivers a working, potentially releasable product increment. This is a complete and tested part of the product.

Ca' Foscari University of Venice

Scrum methodology

- Events (or Ceremonies)
  - **Sprint**: A fixed period, typically 1 to 4 weeks, during which the team works to complete a specific set of requirements from the Product Backlog. Each sprint ends with a working product increment.
  - **Sprint Planning**: At the beginning of each sprint, the team and Product Owner meet to define what will be achieved and how the team plans to accomplish it.
  - **Daily Scrum**: A brief, 15-minute daily meeting where the team updates each other on what has been done, what will be done next, and any impediments.
  - **Sprint Review**: At the end of each sprint, the team presents the completed work to stakeholders to gather feedback. This event helps ensure the product remains aligned with customer needs.
  - **Sprint Retrospective**: Following the Sprint Review, the team holds a meeting to reflect on what went well and what can be improved for the next sprint. It fosters continuous improvement.

- Sprint goal
  - Each sprint has a specific Sprint Goal, which represents the desired outcome and guides the team's work. This provides direction and clarity on what is prioritized during the sprint.

- The Scrum process is cyclical, with each sprint repeating the same steps. Every sprint begins with planning, where the team selects which features from the Product Backlog will be implemented. During the sprint, the team works on the selected tasks, holding daily meetings to track progress. At the end of the sprint, the team presents the incremented product and reflects on how to improve for the next sprint. Then, a new sprint begins, and the cycle repeats.

Scrum methodology - benefits

- **Flexibility**: Scrum is well-suited for projects that require frequent changes, as requirements can be adapted based on feedback at the end of each sprint.
- **Quick Delivery of Value**: With frequent releases, customers can see progress and provide feedback faster, leading to better outcomes.
- **Collaboration and Transparency**: Cross-functional teams and frequent meetings enhance communication and collaboration.
- **User-Focused**: By continuously incorporating feedback, Scrum ensures that the focus remains on user needs and goals.

In summary, Scrum is a lean and collaborative framework that helps manage complex projects effectively by iterating quickly and ensuring goals and outcomes stay aligned throughout the development process.

MVP (Minimal Value Product) is a concept popularized by Lean Startup.

It is common use starting from a MVP and works with Scrum methodology in order to extend the MVP following the market reaction.

Ca' Foscari University of Venice

Kanban methodology

- Developed by Taiichi Ohno for the Toyota Production System
- Helpful for visualize the work status (it helps in seeing the progress and identifying bottlenecks)
- Moreover, the board some principles were added from the application to the software development

Ca' Foscari University of Venice

Kanban methodology - principles

- Visualize the workflow (using the kanban board)
- Limit Work in Progress (WIP): set limits on the number of task that can be in progress at the same time
- Manage and measure flow to ensure task more smoothy through the various stages of the process
- Make Process Policies Explicit: clearly define the rules and policies that govern the workflow; this includes the criteria for task completion and rules for moving tasks from one stage to another

Ca' Foscari University of Venice

Kanban methodology - principles

- Implement Feedback Loops: Regularly implement feedback loops to continuously improve the process; this can include review meetings and retrospectives
- Collaborate and improve Continuously: foster a culture of collaboration and continuos improvement, where the team works together to optimize the process and solve problems

Kanban methodology – metrics and class of services

- Metrics
  - Cumulative Flow Diagrams (number of tasks in each stage of the workflow at any given time)
  - Control charts for cycle time (the time it takes for a task to move from start to finish)
  - Throughput analysis (the number of tasks completed in a given period)

- Some class of services (categorization of task, based on their priority)
  - Expedite: it needs immediate attention and must be completed asap
  - Fixed Date: it has a specific deadline
  - Standard: regular tasks that follow the normal workflow
  - Intangible: important tasks for long-term improvements, such as maintenance or process optimization

Ca' Foscari University of Venice

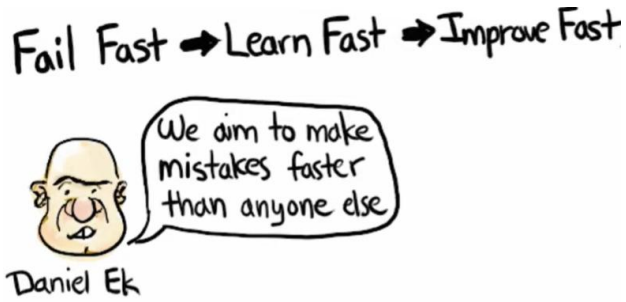
Kanban methodology - benefits

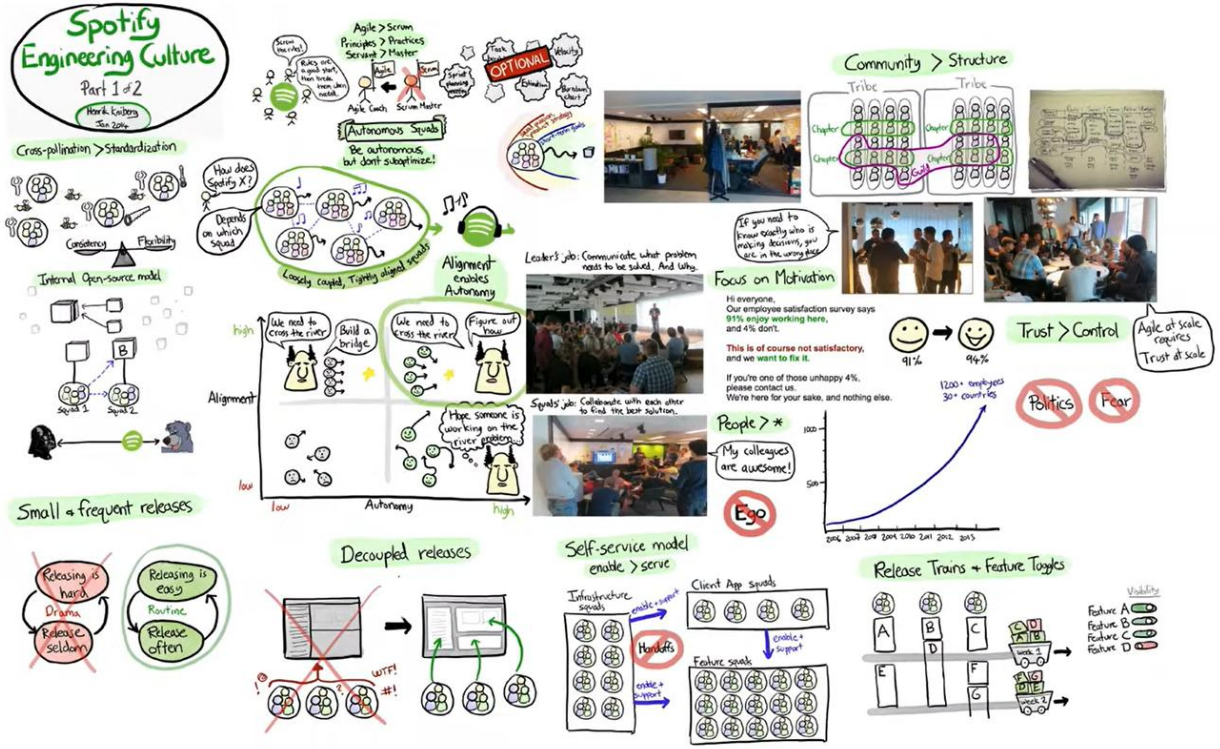
- Improved visibility for task status
- Enhanced efficiency (limiting WIP helps eams focus on completing tasks before starting new ones)
- Flexibility (for different types of work and industries)
- Continuous delivery (Kanban support continuous flow of work)
- Better collaboration (visual nature encourage the collaboration within the team)
- Continuous improvement (regular feedback loops and retrospectives help team to improvement)
- Reduced Cycle Time (managing the flow you can reduce the time it takes to complete tasks, leading a fast delivery)
- Increased predictability (clear view of workflow improve the accuracy for the prediction of the delivery)

Ca' Foscari University of Venice

Real case: Spotify

- Spotify Engineering Culture
  - part 1 (13mins) https://www.youtube.com/watch?v=Yvfz4HGtoPc
  - part 2 (12 mins) https://www.youtube.com/watch?v=vOt4BbWLWQw





Ca' Foscari University of Venice

Extreme Programming methodology

- It is emphasizing customer satisfaction, teamwork and continuous improvement
- Key principles:
  - Communication, open and frequent communication among team members
  - Simplicity, focus on the simplest solution that works
  - Feedback, obtain regular feedback from customers and stakeholders
  - Courage, have the courage to make necessary changes, even if they are difficult (refactoring code, addressing issues promptly, and embracing new ideas)
  - Respect, foster a respectful environment where team members value each other's contributions

Extreme Programmin methodology – core practices

- The Planning
- GameSmall releases
- Metaphor (shared understanding of the system)
- Simple design
- Testing (unit and acceptance)
- Refactoring
- Pair programming
- Collective ownership
- Continuous integration
- 40-hour week
- On-site customer
- Coding standards

Extreme Programmin methodology – software lifecycle

- Planning: invoves creating user stories to define features and functionalities
- Design: emphasizes simple design and continuous refactoring – keep the design as simple as possible
- Coding: devs work in pairs to write code
- Testing: continuous testing is a cornerstone; unit test are written before the code (TDD)
- Integration: frequent integration of code into the main branch to detect issues early
- Release: small frequent releases to deliver value to customers quickly and gather feedback
- Retrospective: after each iteration, the team reflects on what went well and what could be improved

Extreme Programmin methodology – customer role

- Defining requirements: it provides user stories that describe the desired features and functionalities of the product
- Prioritizing work: priority based on business value and urgency
- Providing feedback: it is essential for making adjustments and ensuring the product meets their needs
- Acceptance testing: it is involved in writing and executing acceptance tests to verify that the product meets the requirements
- Decision making: the customer makes key decisions regarding scope, features and timelines
- Collaboration: it collaborates closely with the development team, attending planning meetings, iteration reviews and retrospectives

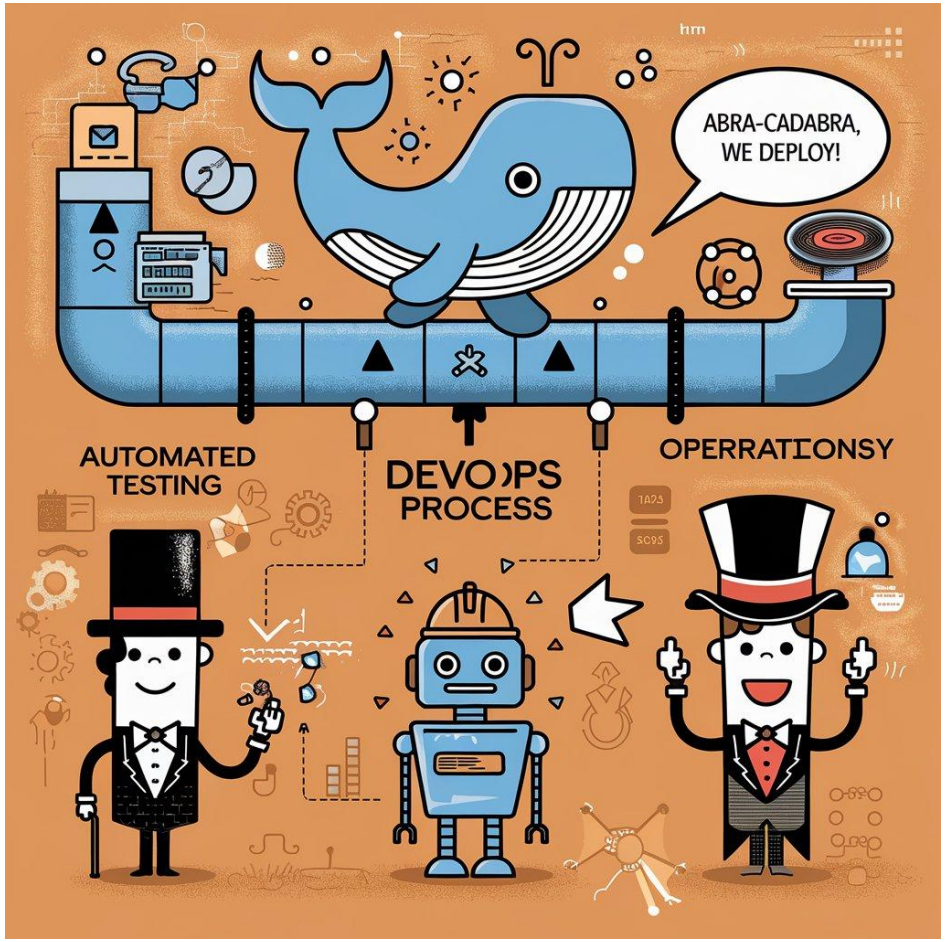
Ca' Foscari University of Venice

Extreme Programmin methodology – benefits

- High-quality code: pair programming, TDD and continuous integration
- Customer satisfaction: customer work closely with the team
- Flexibility and adaptability: changing requirements and priorities fast, making the methodology suitable for dynamic and evolving projects
- Improved communication: regular feedback loops, daily stand-ups and close collaboration between team members
- Increased Productivity: distribution of knowledge accross the team reduce bottlenecks and increasing overall productivity
- Continuous improvement: regular retrospectives and continuous feedback, fostering a culture of ongoing improvement and learning
- Reduce risk: delivery small, incremental updates and continuously testing the code
- Enhanced team morale: collaborative and supportive environment

Ca' Foscari University of Venice

DevOps methodology



- It is a set of practice that combines software development (Dev) and IT operation (Ops)
- Aims to shorten the systems development life cycle and provide continuous delivery with high software quality
- it is not:
  - just a set of tools
  - a specific job title or role
  - limited to startups or web companies

DevOps methodology

- Cultural shift
  - Emphasizes collaboration and communication between development and operation teams
  - Promotes a culture of shared responsibility
  - Encourages cross-functional teams
  - Eliminates traditional barriers between development, operations and quality assurance
  - Align technical projects with business objectives
- Key benefits
  - Faster time to market
  - Improved product quality
  - Increase customer satisfaction
  - Better team collaboration and productivity

DevOps methodology – core principles

- Continuous Integration (CI): frequent merging of code changes into a central repository, automated building and testing of code and early detection of integration issues
- Continuous Delivery (CD): automating the process of software delivery to production, ensuring code is always in a deployable state and reducing time between writing code and its deployment
- Continuous Deployment: Automatically deploying every change that passes all stages of production pipeline, eliminating manual steps in software delivery and enabling rapid and frequent releases
- Automation: automating repetitive tasks, (e.g., build, test and deployment process) and freeing up time for innovation and problem-solving

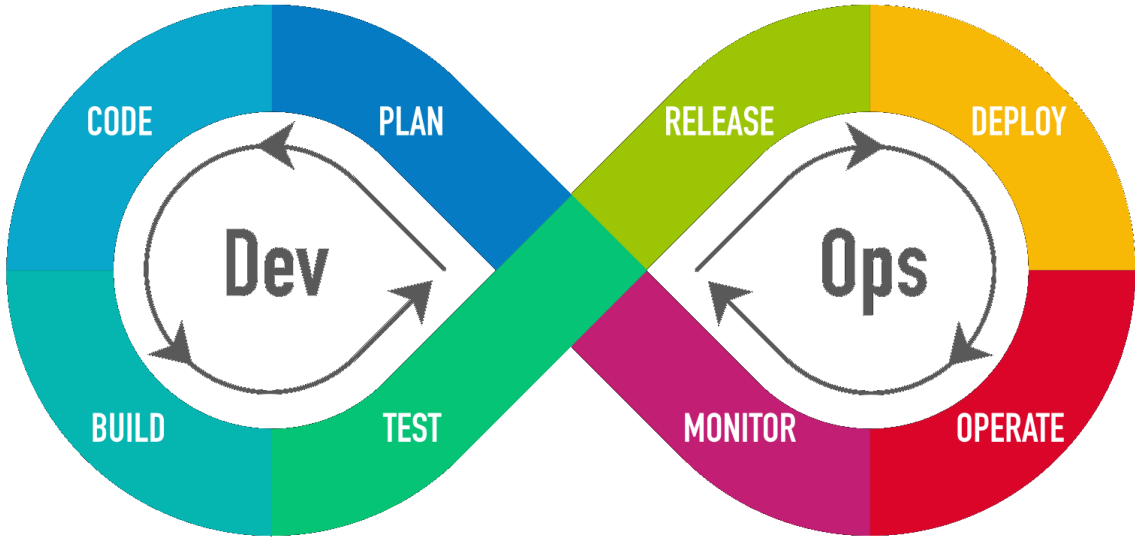
DevOps methodology – core principles

- Monitoring and Logging: real-time monitoring of application and infrastructure performance, centralized logging for easier troubleshooting and proactive identification of issues before impact users
- Infrastructure as Code (IaC): managing and provisioning infrastructure through code instead of manual process, version controlling infrastructure configurations and ensuring consistency across different environments

Ca' Foscari University of Venice

DevOps methodology – lifecycle stages

- Plan – define features, requirements and fixes
- Code – develop and review code
- Build – compile code and create deployable artifacts
- Test – conduct automated testing
- Release – prepare the release and approve for deployment
- Deploy – deploy the application to production
- Operate – manage the application and infrastructure
- Monitor – collect and analyze data on application performance and user behavior



DevOps methodology – key practices and metrics

key practices

- Version control
- Automated testing
- Continuous integration pipelines
- Deployment automation
- Configuration Management
- Containerization (packaging application and dependencies into containers and ensuring consistency across development, testing and production environments)
- Orchestration (managing container deployment and scaling, automating container lifecycle management)
- Microservices architecture

key metrics

- Deployment Frequency: How often an organization deploys code
- Lead Time for Changes: Time from code commit to code running in production
- Mean Time to Recovery (MTTR): How long it takes to recover from a failure
- Change Failure Rate: Percentage of deployments causing a failure in production

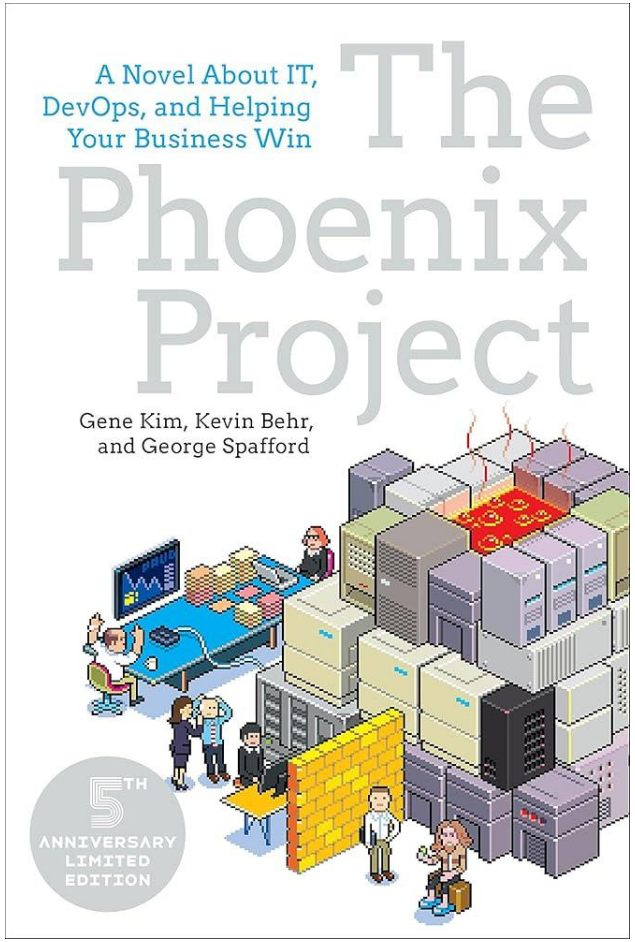
Ca' Foscari University of Venice

DevOps methodology - benefits

- Improved Collaboration and Communication - DevOps fosters a culture of collaboration between development (Dev) and operations (Ops) teams. This breaks down traditional silos, allowing for seamless communication and teamwork.
- Faster and More Frequent Releases - DevOps enables faster development cycles and more frequent software releases. Practices like Continuous Integration (CI) and Continuous Deployment (CD) automate much of the process, reducing time to market.
- Increased Efficiency Through Automation - Many repetitive tasks, such as testing, integration, and deployment, are automated in a DevOps environment, reducing human error and freeing up teams to focus on higher-value activities.
- Enhanced Quality and Reliability - DevOps encourages early detection of bugs through automated testing and continuous monitoring, leading to more stable and reliable systems.
- Scalability and Flexibility - DevOps practices allow systems to scale more easily to meet demand, with infrastructure-as-code (IaC) making it simpler to manage and deploy infrastructure changes.
- Shorter Time to Recovery - In the case of system failures, DevOps practices like continuous monitoring and automated rollback processes help teams recover more quickly, minimizing downtime.
- Customer-Centric Approach - By enabling rapid iterations and feedback loops, DevOps helps deliver features and updates that better meet customer needs, enhancing user satisfaction and competitiveness.

Ca' Foscari University of Venice

The Phoenix Project



- The concept of the "Three ways" are explicity discussed throughout the story of the company's transformation
- 1. The Flow (The First Way)
  - The first principle focuses on the flow of work from development (Dev) to operations (Ops). It emphasizes continuously improving workflow, reducing waste, and eliminating bottlenecks that slow down the release of new software. This concept is essential to DevOps: accelerating delivery times while maintaining high product quality.
- 2. Feedback Loops (The Second Way)
  - The second principle is about creating fast feedback loops between development and operations teams, as well as including customers. This constant cycle of feedback helps identify and fix issues quickly, improving software quality and responsiveness to changes or problems encountered in production.
- 3. Continuous Learning and Improvement (The Third Way)
  - The third principle highlights the importance of a culture of continuous learning and experimentation. Instead of aiming for perfect releases on the first try, DevOps encourages constant learning from both failures and successes. This is reflected in a culture that embraces mistakes and focuses on automation, reducing the risk of large-scale failures.

Examples of companies that use devops methodology

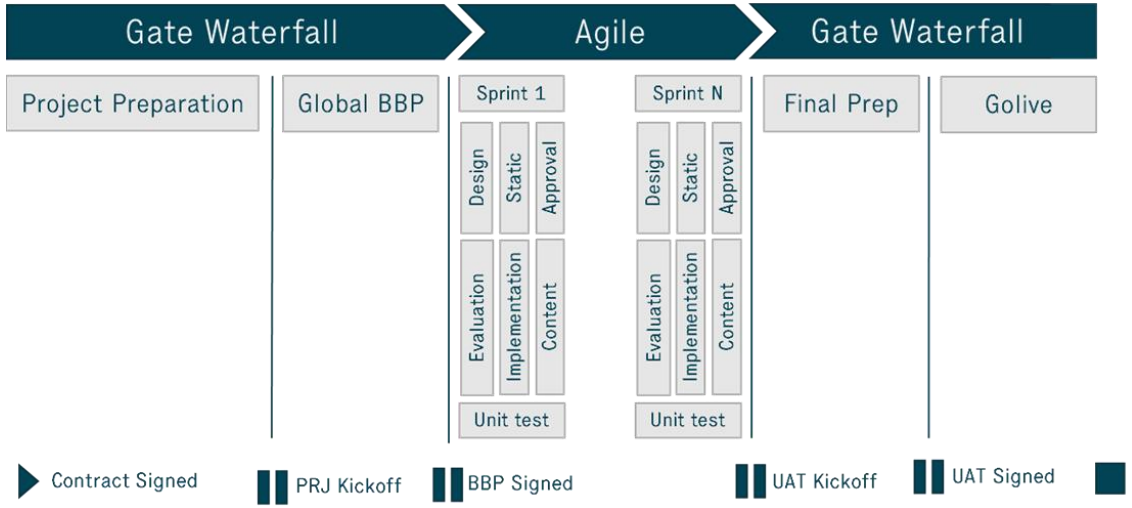
- Video streaming based on complex infrastructure
- It uses the DevOps methodology to maintain a high level of reliability and availability of its services
- Chaos Monkey - https://netflix.github.io/chaosmonkey/ for "failure as a service"

NETFLIX

Examples of companies that use devops methodology

- Software projects required high precision and no errors
- For Mars mission, devops methodology was used for Rover's software update, using cloud infrastructure for simulation of Mars environment
- It achieves an increase in quality by reducing the time of the releases

Ca' Foscari University of Venice

Conclusion

- Pure methodology works, but not always
- In the most of case, some hybrid model works better
- The main point consists of analyzing your goals and define the right way to design the process – considering all the constraints